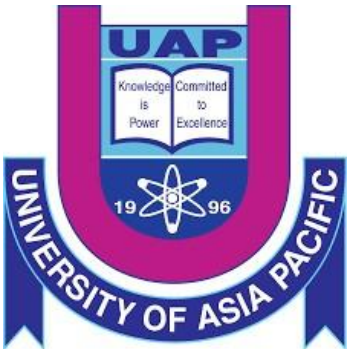



# Report: *2 Tic-Tac-Toe Game*

Course Code: CSE 404

Course Title: Artificial Intelligence and Expert Systems Lab

Submitted to:
Nahida Marzan
Lecturer
Department of CSE

Submitted by:
Name: Rabia Sultana Aunti
ID: 20201019

## 1. Problem Title

Two-Player Console Tic Tac Toe Game in Python

## 2. Problem Description

- Tic Tac Toe is a two-player game played on a 3×3 grid using X and O.
- Players take turns, and the goal is to make three same symbols in a row, column, or diagonal.
- The game is designed as an adversarial search problem where one player tries to win and the other tries to stop it.
- An AI player is developed using the minimax algorithm to choose the best move.
- Alpha-beta pruning is used to improve efficiency by reducing unnecessary calculations.
- The program runs in the console and supports Human vs Computer and Computer vs Computer modes.
- The board is printed after every move, and the game ends with a win or a draw.

## 3. Tools and Languages Used

**Programming Language:** Python Standard
**Libraries:**

- OS – to clear the screen so Player 2 cannot see the secret word.
- set and string operations – to track guessed letters and validate input.

**Platform:** Any operating system that supports Python (Windows).
**Editor/IDE:** Colab.

## 4. Game Flowchart

Start
↓
Initialize empty board
↓
Display position guide (1–9)
↓
Select game mode
(Human vs Computer / Computer vs Computer)
↓
Assign symbols (X and O)
↓
Set search depth for AI
↓
Set current player = X
↓
WHILE game not over:

↓

Print current player

↓

IF Human turn:

Take input move

Validate move

ELSE (AI turn):

Use Minimax with Alpha-Beta

Select best move

↓

Update board

↓

Print board

↓

Check winner?

YES → Display winner → END

↓

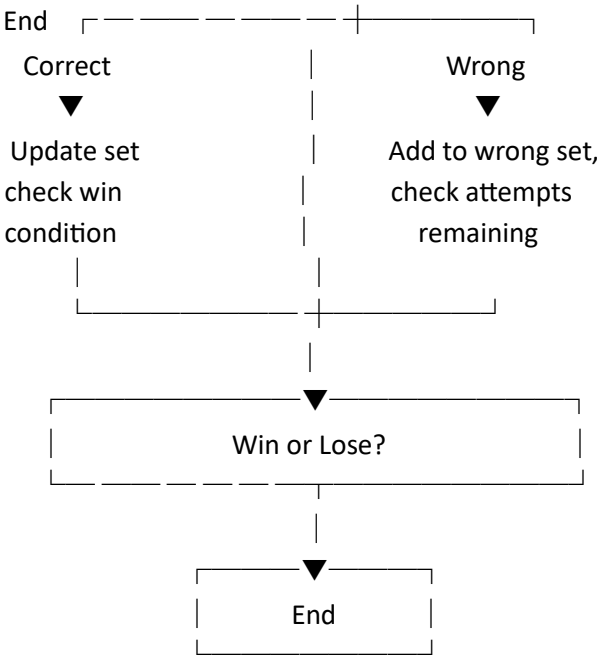
Check draw?

YES → Display draw → END

↓

Switch player (X ↔ O)

↓



## 5. Sample Input/Output (Gameplay Snapshot)

The simulation was implemented in Python.

**Source Code:**

```python
import math
class TicTacToe:
    def __init__(self):
        self.board = [" " for _ in range(9)]

    def print_board(self):
        print()
        for i in range(3):
            row = self.board[i * 3:(i + 1) * 3]
            print(" " + " | ".join(row))
            if i < 2:
                print("---+---+---")
        print()

    def print_positions(self):
        print()
        print("Position numbers:")
        for i in range(3):
            row = [str(i * 3 + j + 1) for j in range(3)]
            print(" " + " | ".join(row))
            if i < 2:
                print("---+---+---")
        print()

    def available_moves(self):
        moves = []
        for i in range(9):
            if self.board[i] == " ":
                moves.append(i)
        return moves

    def make_move(self, index, player):
        if 0 <= index < 9 and self.board[index] == " ":
            self.board[index] = player
            return True
        return False

    def undo_move(self, index):
        self.board[index] = " "

```

```
37
38      def undo_move(self, index):
39          self.board[index] = " "
40
41      def is_winner(self, player):
42          win_patterns = [
43              [0, 1, 2],
44              [3, 4, 5],
45              [6, 7, 8],
46              [0, 3, 6],
47              [1, 4, 7],
48              [2, 5, 8],
49              [0, 4, 8],
50              [2, 4, 6]
51          ]
52
53          for pattern in win_patterns:
54              if (
55                  self.board[pattern[0]] == player and
56                  self.board[pattern[1]] == player and
57                  self.board[pattern[2]] == player
58              ):
59                  return True
60          return False
61
62      def is_draw(self):
63          return " " not in self.board and not self.is_winner("X") and not self.is_winner("O")
64
65      def game_over(self):
66          return self.is_winner("X") or self.is_winner("O") or self.is_draw()
67
68      def evaluate(self, ai_player, human_player, depth):
69          if self.is_winner(ai_player):
70              return 10 - depth
71          if self.is_winner(human_player):
72              return depth - 10
73          return 0
74
75      def minimax(self, depth, is_maximizing, ai_player, human_player, alpha, beta, max_depth):
76          score = self.evaluate(ai_player, human_player, depth)
77
```

[1]
✓ 3m

```
77
78          if score != 0:
79              return score
80
81          if self.is_draw():
82              return 0
83
84          if max_depth != -1 and depth >= max_depth:
85              return 0
86
87          if is_maximizing:
88              best_score = -math.inf
89              for move in self.available_moves():
90                  self.make_move(move, ai_player)
91                  value = self.minimax(depth + 1, False, ai_player, human_player, alpha, beta, max_depth)
92                  self.undo_move(move)
93
94                  if value > best_score:
95                      best_score = value
96
97                  if best_score > alpha:
98                      alpha = best_score
99
100                 if beta <= alpha:
101                     break
102
103             return best_score
104         else:
105             best_score = math.inf
106             for move in self.available_moves():
107                 self.make_move(move, human_player)
108                 value = self.minimax(depth + 1, True, ai_player, human_player, alpha, beta, max_depth)
109                 self.undo_move(move)
110
111                 if value < best_score:
112                     best_score = value
113
114                 if best_score < beta:
115                     beta = best_score
116
117                 if beta <= alpha:
```

[1] ✓ 3m

```
                if beta <= alpha:
                    break

            return best_score

    def best_move(self, ai_player, human_player, max_depth):
        best_score = -math.inf
        best_move_index = None

        for move in self.available_moves():
            self.make_move(move, ai_player)
            score = self.minimax(0, False, ai_player, human_player, -math.inf, math.inf,
            self.undo_move(move)

            if score > best_score:
                best_score = score
                best_move_index = move

        return best_move_index


def get_depth_input(player_name):
    while True:
        value = input(f"Enter search depth for {player_name} (-1 for full search): ").str
        try:
            depth = int(value)
            if depth >= -1:
                return depth
            else:
                print("Please enter -1 or a non-negative integer.")
        except ValueError:
            print("Invalid input. Please enter an integer.")


def human_turn(game, player):
    while True:
        move = input(f"Player {player}, enter your move (1-9): ").strip()
        try:
            move = int(move) - 1
            if game.make_move(move, player):
```



```
[1]
✓ 3m
                move = int(move) - 1
                if game.make_move(move, player):
                    break
                else:
                    print("Invalid move. Try again.")
            except ValueError:
                print("Please enter a number from 1 to 9.")


def ai_turn(game, ai_player, opponent_player, depth):
    move = game.best_move(ai_player, opponent_player, depth)
    if move is not None:
        game.make_move(move, ai_player)
        print(f"Computer ({ai_player}) chooses position {move + 1}")


def play_human_vs_computer():
    game = TicTacToe()
    game.print_positions()

    while True:
        human = input("Choose your symbol (X/O): ").strip().upper()
        if human in ["X", "O"]:
            break
        print("Please choose X or O.")

    computer = "O" if human == "X" else "X"
    depth = get_depth_input("Computer")

    current_player = "X"

    while not game.game_over():
        print(f"Current turn: {current_player}")

        if current_player == human:
            human_turn(game, human)
        else:
            ai_turn(game, computer, human, depth)

        game.print_board()

        if game.is_winner(current_player):
```

[1]
✓ 3m

```
        game.print_board()

        if game.is_winner(current_player):
            if current_player == human:
                print("Human wins!")
            else:
                print("Computer wins!")
            return

        if game.is_draw():
            print("Game ended in a draw.")
            return

        current_player = "O" if current_player == "X" else "X"


def play_computer_vs_computer():
    game = TicTacToe()
    game.print_positions()

    depth_x = get_depth_input("Computer X")
    depth_o = get_depth_input("Computer O")

    current_player = "X"

    while not game.game_over():
        print(f"Current turn: {current_player}")

        if current_player == "X":
            ai_turn(game, "X", "O", depth_x)
        else:
            ai_turn(game, "O", "X", depth_o)

        game.print_board()

        if game.is_winner(current_player):
            print(f"Computer {current_player} wins!")
            return

        if game.is_draw():
            print("Game ended in a draw.")
```

[1]
✓ 3m

```
        while not game.game_over():
            print(f"Current turn: {current_player}")

            if current_player == "X":
                ai_turn(game, "X", "O", depth_x)
            else:
                ai_turn(game, "O", "X", depth_o)

            game.print_board()

            if game.is_winner(current_player):
                print(f"Computer {current_player} wins!")
                return

            if game.is_draw():
                print("Game ended in a draw.")
                return

            current_player = "O" if current_player == "X" else "X"

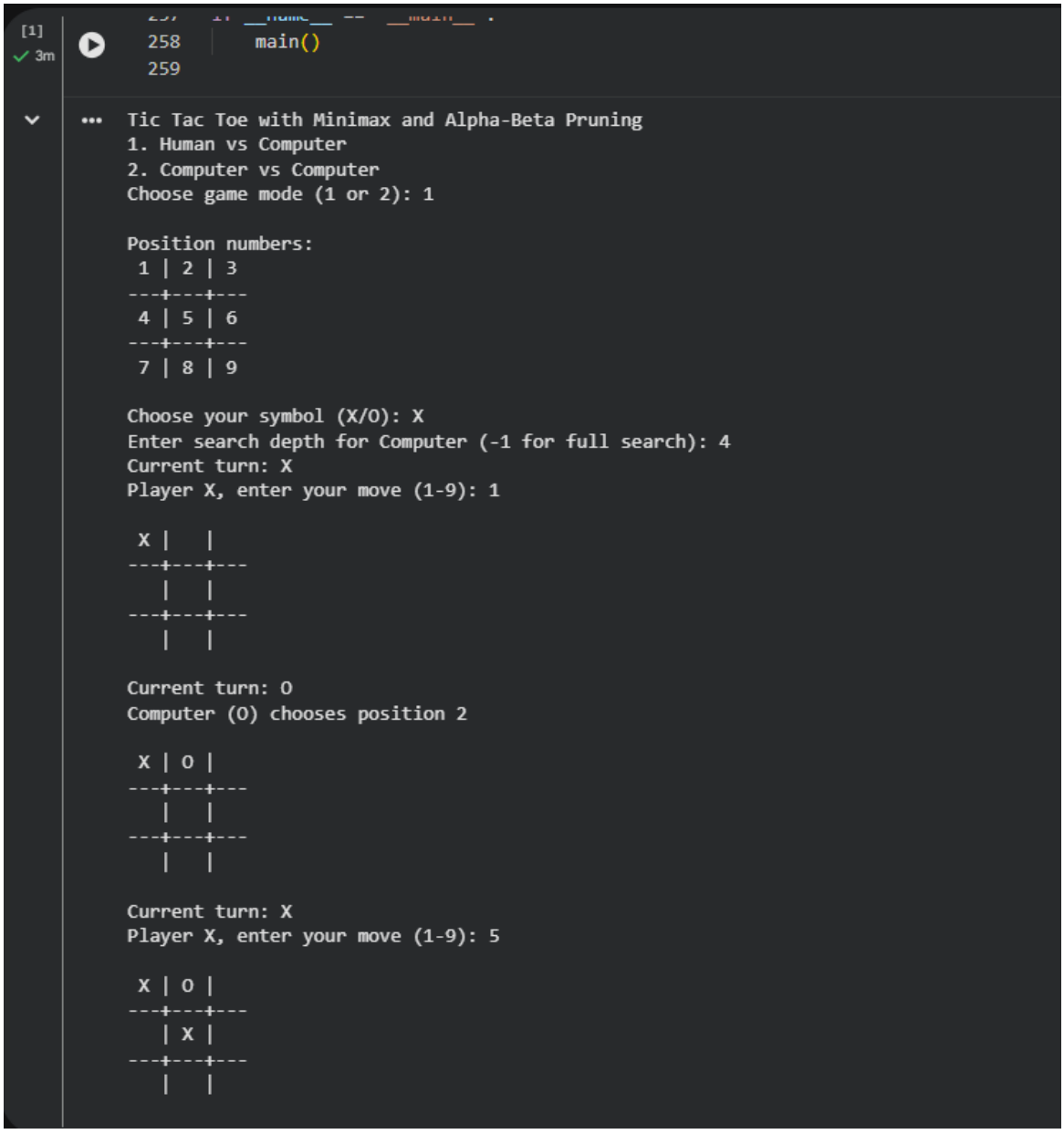

    def main():
        print("Tic Tac Toe with Minimax and Alpha-Beta Pruning")
        print("1. Human vs Computer")
        print("2. Computer vs Computer")

        while True:
            choice = input("Choose game mode (1 or 2): ").strip()
            if choice == "1":
                play_human_vs_computer()
                break
            elif choice == "2":
                play_computer_vs_computer()
                break
            else:
                print("Invalid choice. Please enter 1 or 2.")


    if __name__ == "__main__":
        main()

```

**Output:**



```
[1]
✓ 3m
258     main()
259

Tic Tac Toe with Minimax and Alpha-Beta Pruning
1. Human vs Computer
2. Computer vs Computer
Choose game mode (1 or 2): 1

Position numbers:
 1 | 2 | 3
---+---+---
 4 | 5 | 6
---+---+---
 7 | 8 | 9

Choose your symbol (X/O): X
Enter search depth for Computer (-1 for full search): 4
Current turn: X
Player X, enter your move (1-9): 1

 X |   |
---+---+---
   |   |
---+---+---
   |   |

Current turn: O
Computer (O) chooses position 2

 X | O |
---+---+---
   |   |
---+---+---
   |   |

Current turn: X
Player X, enter your move (1-9): 5

 X | O |
---+---+---
   | X |
---+---+---
   |   |
```

```
   | X |
---+---+---
   |   |

Current turn: O
Computer (O) chooses position 9

 X | O |
---+---+---
   | X |
---+---+---
   |   | O

Current turn: X
Player X, enter your move (1-9): 4

 X | O |
---+---+---
 X | X |
---+---+---
   |   | O

Current turn: O
Computer (O) chooses position 3

 X | O | O
---+---+---
 X | X |
---+---+---
   |   | O

Current turn: X
Player X, enter your move (1-9): 7

 X | O | O
---+---+---
 X | X |
---+---+---
 X |   | O

Human wins!
```

## 6. Conclusion and Challenges

**Conclusion:**
In this project, the Tic tac Toe game was successfully implemented using the minimax algorithm and alpha-beta punning. The AI player can make smart and optimal decisions based on the current game state. If full search is used, the AI will never lose.
The program is simple easy to use, and works correctly in both human vs Computer and Computer vs computer modes. This project helped in understanding game logic, recursion, and decision-making algorithm.

**Challenges:**

1. Understanding the minimax algorithm was difficult at first, especially how recursion works.
2. Implementing alpha-beta pruning correctly was challenging, because wrong placement can give wrong results.
3. Managing board states using undo moves was important to avoid errors during recursion.
4. Handling user input properly (like invalid moves or wrong input) was also a challenge.
5. Choosing the right depth for the AI was tricky, because low depth gives weaker performance.